

Introduction à Android

Moustapha Diouf FALL

UCAD

2020

Plan de l'exposé

- 1 Organisation de ce cours
- 2 Installation et configuration des outils (windows)
- 3 Notre première application

- Nous utiliserons java et Android studio. Android Studio est un environnement de développement spécialisé dans le développement d'applications android et un outil pour gérer l'installation du SDK Android sur votre système.

- Nous utiliserons java et Android studio. Android Studio est un environnement de développement spécialisé dans le développement d'applications android et un outil pour gérer l'installation du SDK Android sur votre système.



Prérequis

- 2 Go de mémoire RAM.
- Plus de 1,5 Go d'espace disque pour tout installer
- Vous aurez besoin d'avoir Windows Vista ou plus récent.
- Java Development Kit
- Android studio

Prérequis

- 2 Go de mémoire RAM.
- Plus de 1,5 Go d'espace disque pour tout installer
- Vous aurez besoin d'avoir Windows Vista ou plus récent.
- Java Development Kit
- Android studio

Prérequis

- 2 Go de mémoire RAM.
- Plus de 1,5 Go d'espace disque pour tout installer
- Vous aurez besoin d'avoir Windows Vista ou plus récent.
- Java Development Kit
- Android studio

Prérequis

- 2 Go de mémoire RAM.
- Plus de 1,5 Go d'espace disque pour tout installer
- Vous aurez besoin d'avoir Windows Vista ou plus récent.
- Java Development Kit
- Android studio

Prérequis

- 2 Go de mémoire RAM.
- Plus de 1,5 Go d'espace disque pour tout installer
- Vous aurez besoin d'avoir Windows Vista ou plus récent.
- Java Development Kit
- Android studio

Installation de android studio, jdk et d'un emulateur

Android studio

- Télécharger Android studio sur
[https ://developer.android.com/studio](https://developer.android.com/studio)
- Installer Android studio
- Vérifier les mises à jour.

Installation de android studio, jdk et d'un emulateur

Android studio

- Télécharger Android studio sur
[https ://developer.android.com/studio](https://developer.android.com/studio)
- Installer Android studio
- Vérifier les mises à jour.

Installation de android studio, jdk et d'un emulateur

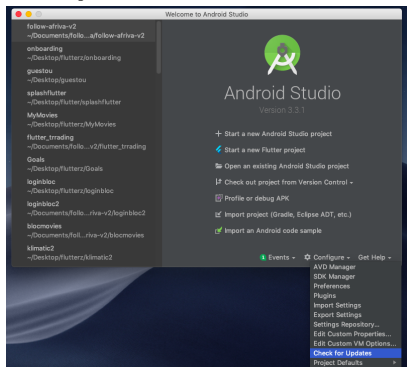
Android studio

- Télécharger Android studio sur [https ://developer.android.com/studio](https://developer.android.com/studio)
- Installer Android studio
- Vérifier les mises à jour.

Installation de android studio, jdk et d'un emulateur

Android studio

- Télécharger Android studio sur <https://developer.android.com/studio>
- Installer Android studio
- Vérifier les mises à jour.



Installation de android studio et d'un emulateur

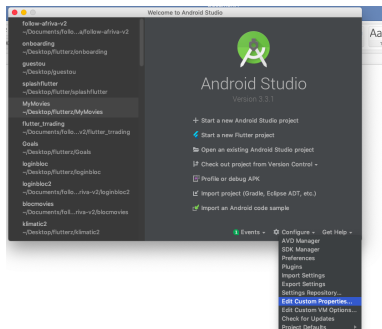
Configuration de android studio

- Créer un dossier pour stocker nos projets
- Demarrer android studio

Installation de android studio et d'un emulateur

Configuration de android studio

- Créer un dossier pour stocker nos projets
- Demarrer android studio



Installation de android studio et d'un emulateur

Configuration de android studio

- Installation d'un SDK (software development kit) : Les applications Android sont développées en Java, mais un appareil sous Android ne comprend pas le Java tel quel, il comprend une variante du Java adaptée pour Android. Un SDK, un kit de développement dans notre langue, est un ensemble d'outils permettant de développer pour une cible particulière
- Ils nous permettent de développer des application d'android pour une version particulière d'Android

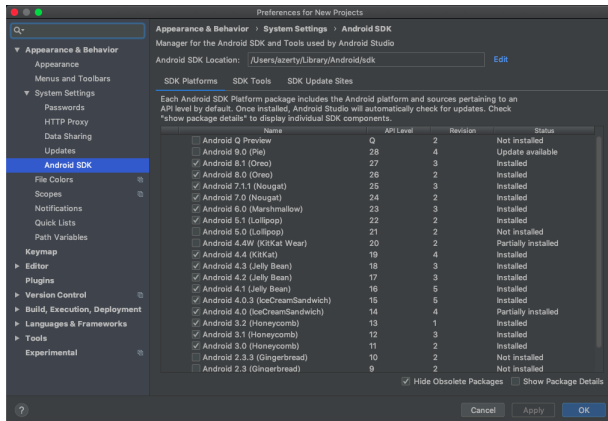
Installation de android studio et d'un emulateur

Configuration de android studio

- Installation d'un SDK (software development kit) : Les applications Android sont développées en Java, mais un appareil sous Android ne comprend pas le Java tel quel, il comprend une variante du Java adaptée pour Android. Un SDK, un kit de développement dans notre langue, est un ensemble d'outils permettant de développer pour une cible particulière
- Ils nous permettent de développer des application d'android pour une version particulière d'Android

Installation de android studio et d'un emulateur

Configuration de android studio



Installation de android studio et d'un emulateur

Installation d'un emulateur

Un émulateur android est un périphérique virtuel Android (AVD, Android Virtual Device) qui représente un périphérique d'android spécifique. Vous pouvez utiliser un émulateur Android en tant que plate-forme cible pour exécuter et tester vos applications Android sur votre PC.

Installation de android studio et d'un emulateur

Installation d'un emulateur

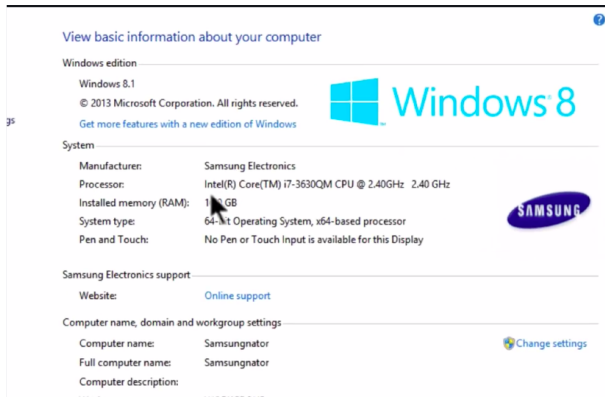
- Vérifier notre CPU.

- Si vous avez un intel votre emulateur sera plus rapide (à continuer)

Installation de android studio et d'un emulateur

Installation d'un emulateur

- Vérifier notre CPU.

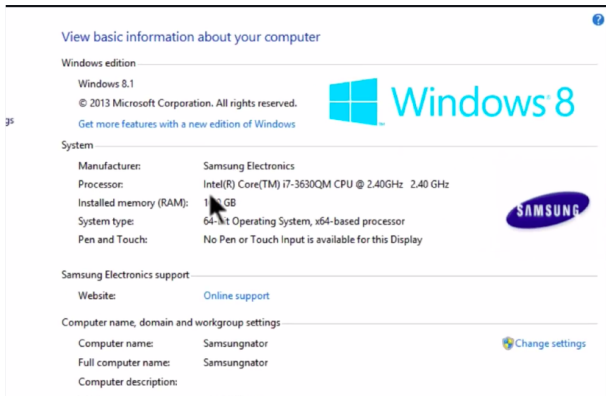


- Si vous avez un intel votre emulateur sera plus rapide (à continuer)

Installation de android studio et d'un emulateur

Installation d'un emulateur

- Vérifier notre CPU.



- Si vous avez un intel votre emulateur sera plus rapide (à continuer)

Astuces pour améliorer les performances d'android studio

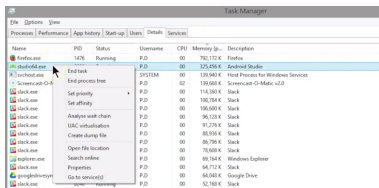
- Fermer les autres programmes dont vous n'avez pas besoin
- Donner à android studio la priorité
- Désactiver temporairement les antivirus

Astuces pour ameliorer les performances d'android studio

- Fermer les autres programmes dont vous n'avez pas besoin
- Donner à android studio la priorité

Astuces pour ameliorer les performances d'android studio

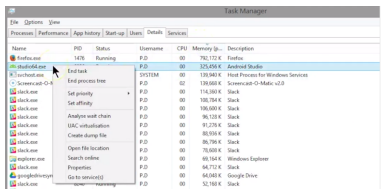
- Fermer les autres programmes dont vous n'avez pas besoin
- Donner à android studio la priorité



- Desactiver temporairement les antivirus

Astuces pour ameliorer les performances d'android studio

- Fermer les autres programmes dont vous n'avez pas besoin
- Donner à android studio la priorité



- Desactiver temporairement les antivirus

Astuces pour améliorer les performances d'android studio

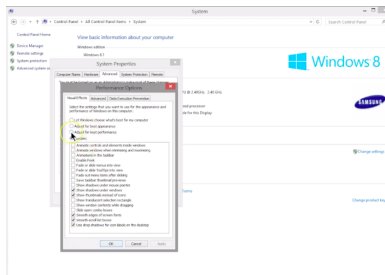
- Désactiver temporairement les antivirus.
- Optimiser windows en désactivant les animations.

Astuces pour améliorer les performances d'android studio

- Désactiver temporairement les antivirus.
- Optimiser windows en désactivant les animations.

Astuces pour ameliorer les performances d'android studio

- Desactiver temporairement les antivirus.
- Optimiser windows en desactivant les animations.



Android est la plateforme mobile la plus populaire au monde. Au dernier décompte, il y avait plus de deux milliards d'appareils Android actifs dans le monde entier, et ce nombre augmente rapidement. Android est une plateforme open source complètement basée sur Linux et soutenu par Google. C'est un puissant cadre de développement qui comprend tout ce dont vous avez besoin pour créer de superbes applications en utilisant un mélange de Java et XML. Il vous permet de déployer ces applications sur une grande variété d'appareils : téléphones, tablettes, etc. Alors, qu'est-ce qui constitue une application Android typique ?

- "Layouts"
- "Activities"

"Layouts"

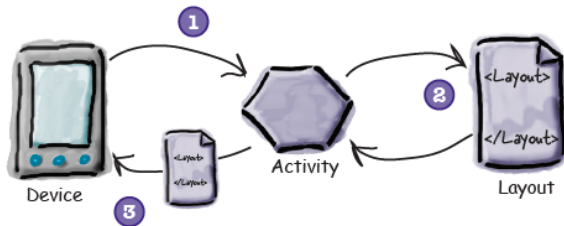
Une application Android typique est composée d'une ou plusieurs écrans. On utilise les "Layouts" pour définir l'apparence de chaque ecran. Les layouts sont généralement définies en XML, et peut inclure des composants GUI tels que sous forme de boutons, champs de texte et étiquettes.

"Activity"

Les Layouts définissent uniquement l'apparence de l'application. Vous définissez ce que l'application fait en utilisant une ou plusieurs "activities". Une activité est une classe Java qui décide quel layout elle doit utiliser et indique à l'application comment répondre à l'utilisateur.

SDK

Java est le langage le plus utilisé pour développer des applications Android. Les appareils Android n'exécutent pas les fichiers .class et .jar. Au lieu de cela, pour améliorer la vitesse et les performances de la batterie, les appareils Android utilisent leur propre format optimisé pour le code compilé. Cela signifie que vous ne pouvez pas utiliser un Environnement de développement Java, vous avez également besoin d'outils spéciaux pour convertir votre code compilé dans un format Android, pour les déployer sur un Appareil Android et pour vous permettre de déboguer l'application une fois qu'elle est en cours d'exécution. Tous ces éléments font partie du SDK Android.



- L'appareil lance votre application et crée une activité.
- L'activité spécifie un layout à utiliser.
- L'activité demande à l'android d'afficher le layout
- L'utilisateur interagit avec le layout qui est affiché sur l'appareil
- L'activité répond à ces interactions par exécution du code d'application.
- L'activité se met à jour l'affichage..
- que l'utilisateur voit sur l'appareil

I am rich App

En 2008, un développeur allemand nommé Armin Heinrich a eu l'idée d'offrir aux utilisateurs de première génération d'iPhone une application dans laquelle il n'y aurait rien, et ce, contre la modique somme de 999,99

I am rich App

En 2008, un développeur allemand nommé Armin Heinrich a eu l'idée d'offrir aux utilisateurs de première génération d'iPhone une application dans laquelle il n'y aurait rien, et ce, contre la modique somme de 999,99

Android est la plateforme mobile la plus populaire au monde. Au dernier décompte, il y avait plus de deux milliards d'appareils Android actifs dans le monde entier, et ce nombre augmente rapidement. Android est une plateforme open source complète basée sur Linux et soutenu par Google. C'est un puissant cadre de développement qui comprend tout ce dont vous avez besoin pour créer de superbes applications en utilisant un mélange de Java et XML. C'est quoi plus, il vous permet de déployer ces applications sur une grande variété de appareils : téléphones, tablettes, etc. Alors, qu'est-ce qui constitue une application Android typique ?

I am rich App

Objectifs

- Nous familiariser avec Android studio
- Comment arranger l'interface graphiques
- Introduction à xml

I am rich App

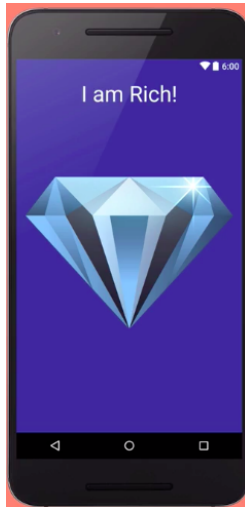
Objectifs

- Nous familiariser avec Android studio
- Comment arranger l'interface graphiques
- Introduction à xml

I am rich App

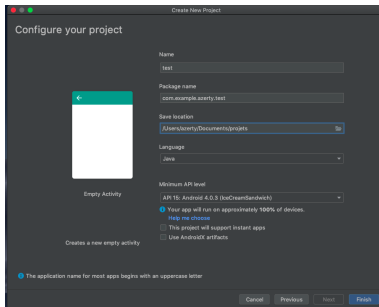
Objectifs

- Nous familiariser avec Android studio
- Comment arranger l'interface graphiques
- Introduction à xml



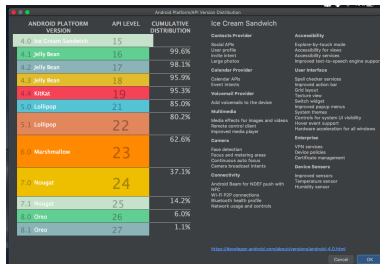
I am rich App

Nouveau projet



I am rich App

Nouveau projet



I am rich App

Composants

Notre projet est constitué de plusieurs fichiers.

- Layout : ce dossier contient les XML de l'application pour les interfaces. Vous pouvez déjà voir le main.xml, qui correspond à l'interface de l'activité principale de l'application.
- Values : ce dossier contient aussi des XML, mais qui servent à stocker des chaînes de caractères.
- AndroidManifest.xml : fichier XML dans lequel on déclare les différentes activités de l'application, et certains paramètres de l'application

I am rich App

Composants

Notre projet est constitué de plusieurs fichiers.

- Layout : ce dossier contient les XML de l'application pour les interfaces. Vous pouvez déjà voir le main.xml, qui correspond à l'interface de l'activité principale de l'application.
- Values : ce dossier contient aussi des XML, mais qui servent à stocker des chaînes de caractères.
- AndroidManifest.xml : fichier XML dans lequel on déclare les différentes activités de l'application, et certains paramètres de l'application

I am rich App

Composants

Notre projet est constitué de plusieurs fichiers.

- Layout : ce dossier contient les XML de l'application pour les interfaces. Vous pouvez déjà voir le main.xml, qui correspond à l'interface de l'activité principale de l'application.
- Values : ce dossier contient aussi des XML, mais qui servent à stocker des chaînes de caractères.
- AndroidManifest.xml : fichier XML dans lequel on déclare les différentes activités de l'application, et certains paramètres de l'application

I am rich App

Les Composants

- Drawable : Ce dossier contient les images de l'application. Tous ces dossiers devraient, dans l'idéal, contenir les mêmes images, mais avec des résolutions différentes. En effet, selon la résolution de l'écran utilisé, les images seront tirées du dossier correspondant.
- MainActivity.java : qui correspond à l'activité principale créée avec le projet (car liée au fichier main.xml)

I am rich App

Les Composants

- Drawable : Ce dossier contient les images de l'application. Tous ces dossiers devraient, dans l'idéal, contenir les mêmes images, mais avec des résolutions différentes. En effet, selon la résolution de l'écran utilisé, les images seront tirées du dossier correspondant.
- MainActivity.java : qui correspond à l'activité principale créée avec le projet (car liée au fichier main.xml

I am rich App

Les widgets de notre application

- Textview : Un textview est utilisé pour afficher du text . .
- ImageView : une imageview est une balise qui possède une image en fond ou un icône visible pour l'utilisateur.

I am rich App

Les widgets de notre application

- Textview : Un textview est utilisé pour afficher du text . .
- ImageView : une imageview est une balise qui possède une image en fond ou un icône visible pour l'utilisateur.

I am rich App

Comprendre le xml et comment ça marche

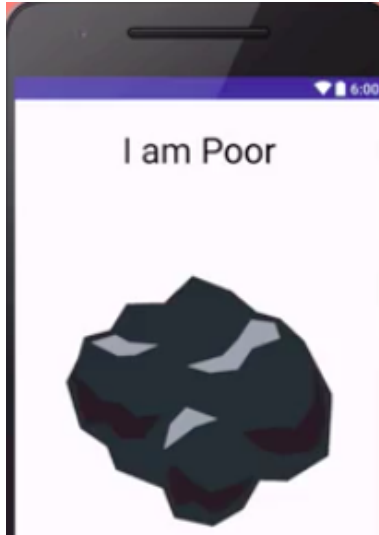
Le XML est un langage de balisage un peu comme le HTML — le HTML est d'ailleurs indirectement un dérivé du XML. Le principe d'un langage de programmation (Java, C++, etc.) est de donner des ordres à l'ordinateur pour qu'il puisse effectuer des calculs, puis éventuellement de mettre en forme le résultat de ces calculs dans une interface graphique.

À l'opposé, un langage de balisage (comme le XML) n'a pas pour objectif de donner des ordres à l'ordinateur, il se contente de définir une façon de mettre en forme l'information de façon à ce qu'on sache comment lire cette information.

Et si on connaît l'architecture d'un fichier, alors il est très facile de retrouver l'emplacement des informations contenues dans ce fichier et de pouvoir les exploiter.

I am rich App challenge

I am poor app



GEO QUIZZ

L'application que vous allez créer s'appelle GeoQuiz. GeoQuiz teste les connaissances de l'utilisateur en matière de géographie. L'utilisateur appuie sur Vrai ou sur Faux pour répondre à la question à l'écran, et GeoQuiz fournit un retour instantané.

GeoQuiz

mettre une image



GeoQuiz

Bases

L'application GeoQuiz comprendra une activity et un layout :

- Une activité est une instance de Activity, une classe du SDK Android. Une activité est responsable de gérer l'interaction de l'utilisateur avec un écran d'informations.
- un Layout définit un ensemble d'objets d'interface utilisateur et leur position à l'écran. un Layout est composée de définitions écrites en XML. Chaque définition est utilisée pour créer un objet qui apparaît à l'écran, comme un bouton ou du texte

GeoQuiz

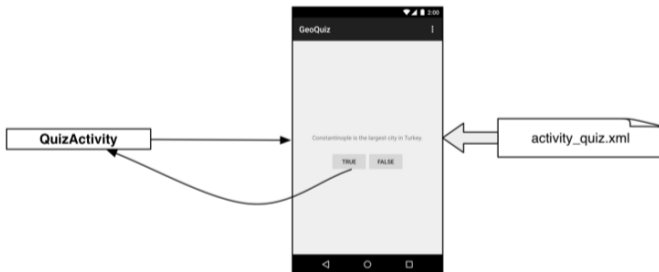
Bases

L'application GeoQuiz comprendra une activity et un layout :

- Une activité est une instance de Activity, une classe du SDK Android. Une activité est responsable de gérer l'interaction de l'utilisateur avec un écran d'informations.
- un Layout définit un ensemble d'objets d'interface utilisateur et leur position à l'écran. un Layout est composée de définitions écrites en XML. Chaque définition est utilisée pour créer un objet qui apparaît à l'écran, comme un bouton ou du texte

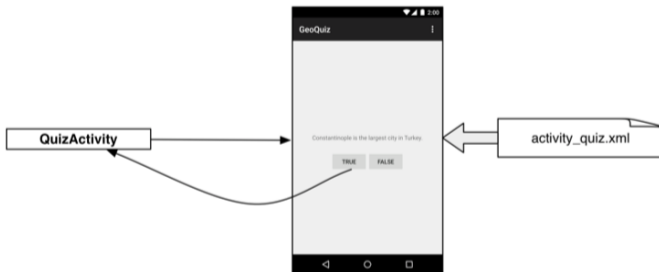
GeoQuiz

Bases



GeoQuiz

Bases



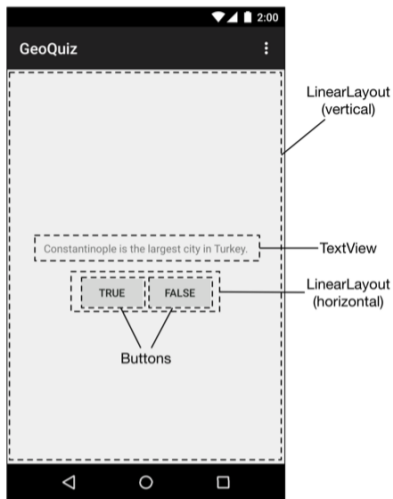
GeoQuiz

Bases

Les widgets sont les blocs que nous utilisons pour composer une interface utilisateur. Un widget peut afficher du texte ou des graphiques, interagir avec l'utilisateur ou organiser d'autres widgets à l'écran. Les boutons, les commandes de saisie de texte et les cases à cocher sont tous des types de widgets. Le SDK Android comprend de nombreux widgets que vous pouvez configurer pour obtenir l'apparence et le comportement souhaités.

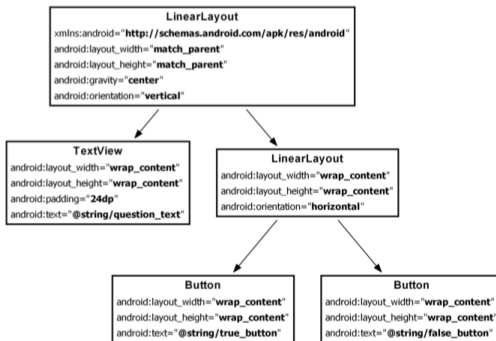
GeoQuiz

Bases



GeoQuiz

Bases



GeoQuiz

Bases

L'élément racine de la hiérarchie de cette présentation est un `LinearLayout`. En tant qu'élément racine, `LinearLayout` doit spécifier l'espace de noms XML de la ressource Android à l'adresse `http://schemas.android.com/apk/res/android`. `LinearLayout` hérite d'une sous-classe de `View` nommée `ViewGroup`. Un `ViewGroup` est un widget qui contient et organise d'autres widgets. Vous utilisez `LinearLayout` lorsque vous souhaitez que les widgets soient organisés dans une seule colonne ou une seule ligne. Les autres sous-classes de `ViewGroup` sont `FrameLayout`, `TableLayout` et `RelativeLayout`. Lorsqu'un widget est contenu par un `ViewGroup`, ce widget est considéré comme un enfant du `ViewGroup`. La racine `LinearLayout` a deux enfants : une `TextView` et une autre `LinearLayout`. L'enfant `LinearLayout` a deux enfants `Button` propres.

GeoQuiz

Bases

- Les attributs android : (layout width) et android : (layout height) sont obligatoires pour presque tous les types de widgets. Ils sont généralement définis sur matchparent ou wrap content : la vue match parent sera aussi grande que son parent
- android :orientation : L'attribut android : orientation sur les deux widgets LinearLayout détermine si leurs enfants apparaîtront verticalement ou horizontalement. La racine LinearLayout est verticale ; son enfant LinearLayout est horizontal. L'ordre dans lequel les enfants sont définis détermine l'ordre dans lequel ils apparaissent à l'écran. Dans un LinearLayout vertical, le premier enfant défini apparaîtra en premier. Dans un LinearLayout horizontal, le premier enfant défini sera le plus à gauche

GeoQuiz

Bases

- Les attributs android : (layout width) et android : (layout height) sont obligatoires pour presque tous les types de widgets. Ils sont généralement définis sur matchparent ou wrap content : la vue match parent sera aussi grande que son parent
- android :orientation : L'attribut android : orientation sur les deux widgets LinearLayout détermine si leurs enfants apparaîtront verticalement ou horizontalement. La racine LinearLayout est verticale ; son enfant LinearLayout est horizontal. L'ordre dans lequel les enfants sont définis détermine l'ordre dans lequel ils apparaissent à l'écran. Dans un LinearLayout vertical, le premier enfant défini apparaîtra en premier. Dans un LinearLayout horizontal, le premier enfant défini sera le plus à gauche

GeoQuiz

RelativeLayout

Un `RelativeLayout` vous permet de positionner les vues par rapport à leur parent ou par rapport aux autres vues de la présentation. Vous définissez une disposition relative à l'aide de l'élément `<RelativeLayout>`

GeoQuiz

Bases

- La méthode `onCreate (Bundle)` est appelée lorsqu'une instance de la sous-classe d'activité est créée. Lorsqu'une activité est créée, elle nécessite une interface utilisateur à gérer. Pour obtenir l'activité de son interface utilisateur, vous appelez la méthode `Activity` suivante : `public void setContentView (int layoutResID)` Cette méthode place un layout à l'écran. Lorsqu'une disposition est placée à l'écran, chaque widget du fichier de disposition est instancié comme défini par ses attributs.

GeoQuiz

Bases

- Un layout est une ressource. Une ressource est un élément de votre application qui n'est pas du code - des éléments tels que des fichiers image, des fichiers audio et des fichiers XML.

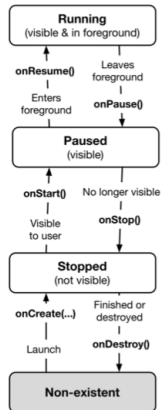
LIFE CYCLE

Bases

Chaque instance d'une activité a un cycle de vie. Pendant ce cycle de vie, une activité effectue une transition entre trois états : en cours d'exécution, en pause et à l'arrêt. Pour chaque transition, il existe une méthode Activity qui notifie l'activité du changement d'état.

GeoQuiz

Bases



GeoQuiz

Bases

Nous connaissons déjà l'une de ces méthodes - onCreate (Bundle). Le système d'exploitation appelle cette méthode après la création de l'instance d'activité, mais avant son affichage à l'écran.

Life cycle

Bases

Généralement, onCreate (...) prépare l' interface utilisateur :

- affiche les widgets à l'écran (dans l'appel à (setContentView (int))
- configurer les listeners sur des widgets pour gérer les interactions de l'utilisateur

Il est important de comprendre que vous n'appellez jamais onCreate (...) ni aucune des autres méthodes de cycle de vie d'Activity. Vous les remplacez dans vos sous-classes d'activité et Android les appelle au moment opportun.

Life cycle

Bases

Généralement, `onCreate (...)` prépare l' interface utilisateur :

- affiche les widgets à l'écran (dans l'appel à `(setContentView (int))`)
- configurer les listeners sur des widgets pour gérer les interactions de l'utilisateur

Il est important de comprendre que vous n'appellez jamais `onCreate (...)` ni aucune des autres méthodes de cycle de vie d'`Activity`. Vous les remplacez dans vos sous-classes d'activité et Android les appelle au moment opportun.

LIFE CYCLE

Making log messages

Dans Android, la classe `android.util.Log` envoie des messages de journal. Le journal a plusieurs méthodes pour consigner les messages. Voici celui que nous utiliserons le plus souvent : `public static int d (balise String, String msg)` Le `d` signifie «débogage» et fait référence au niveau du message de journal. Le premier paramètre identifie la source du message et le second, le contenu du message. La première chaîne est généralement une constante TAG avec le nom de la classe comme valeur. Cela facilite la détermination de la source d'un message particulier.

Rotation and the Activity Lifecycle

Making log messages

Dans Android, la classe `android.util.Log` envoie des messages de journal. Le journal a plusieurs méthodes pour consigner les messages. Voici celui que nous utiliserons le plus souvent : `public static int d (balise String, String msg)` Le `d` signifie «débogage» et fait référence au niveau du message de journal. Le premier paramètre identifie la source du message et le second, le contenu du message. La première chaîne est généralement une constante TAG avec le nom de la classe comme valeur. Cela facilite la détermination de la source d'un message particulier.

Landscape layout

Making log messages

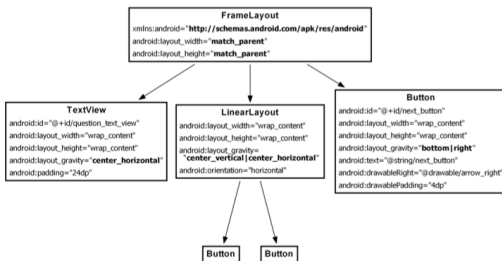
La rotation de l'appareil change la configuration de l'appareil. La configuration de l'appareil est un ensemble de caractéristiques décrivant l'état actuel d'un appareil individuel. Les caractéristiques qui composent la configuration incluent l'orientation de l'écran, la densité de l'écran, la taille de l'écran, le type de clavier, le mode d'ancrage, la langue, etc.

Landscape layout

Making log messages

Le suffixe -land est un autre exemple de qualificatif de configuration. Les qualificateurs de configuration dans les sous-répertoires res indiquent comment Android identifie les ressources correspondant le mieux à la configuration actuelle du périphérique. Vous pouvez trouver la liste des qualificateurs de configuration reconnus par Android et les éléments de la configuration de l'appareil auxquels ils font référence à l'adresse <http://developer.android.com/guide/topics/resources/supply-resources.html>. Lorsque l'appareil est en orientation paysage, Android recherche et utilise les ressources du répertoire res / layout-land. Sinon, cela restera avec la valeur par défaut dans res / layout /.

Landscape



Landscape

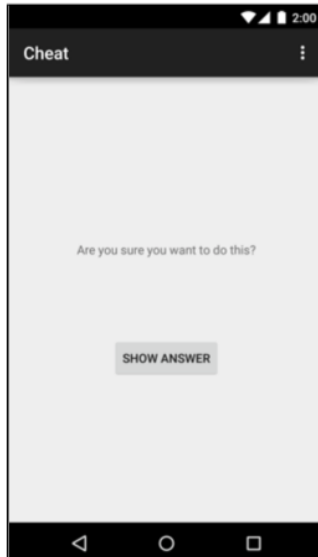
FrameLayout remplacera LinearLayout. FrameLayout est le ViewGroup le plus simple et n'organise pas ses enfants de manière particulière. Dans cette présentation, les vues enfant seront organisées en fonction de leurs attributs android : layout gravity. Les enfants TextView, LinearLayout et Button de FrameLayout ont besoin des attributs android : layout gravity. Les enfants Button de LinearLayout resteront exactement les mêmes.

Enregistrement de données en rotation

Pour résoudre ce problème, QuizActivity post-rotation doit connaître l'ancienne valeur de CurrentIndex. Vous avez besoin d'un moyen de sauvegarder ces données lors d'un changement de configuration d'exécution, tel que la rotation. Une façon de faire est de remplacer la méthode override :

```
protected void onSaveInstanceState(Bundle outState)
```

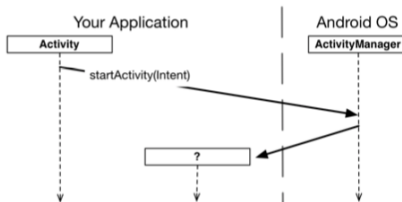

CheatActivity



Commencer une activité

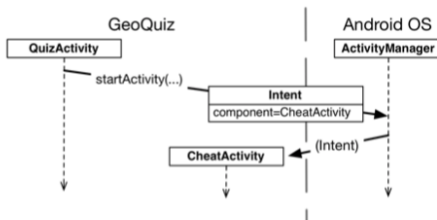
Le moyen le plus simple de commencer une activité avec une autre est d'utiliser la méthode `Activity : public void startActivity (Intent Intent)` Lorsqu'une activité appelle `startActivity (...)`, cet appel est envoyé au système d'exploitation. En particulier, il est envoyé à une partie du système d'exploitation appelée `ActivityManager`. `ActivityManager` crée ensuite l'instance `Activity` et appelle son `onCreate (...)`.

CheatActivity



Un intent est un objet qu'un composant peut utiliser pour communiquer avec le système d'exploitation.

CheatActivity



CheatActivity

Avant de commencer l'activité, `ActivityManager` vérifie le manifeste pour une déclaration portant le même nom que la classe spécifiée. S'il trouve une déclaration, il commence l'activité et tout va bien. Lorsque vous créez un intent avec un contexte et un objet de classe, vous créez un intent explicite. Vous utilisez des intent explicites pour démarrer des activités dans votre application.

AppBar

L "AppBar" est généralement en haut de chaque écran d'une application, qui fournit un aspect familier cohérent entre les applications Android. Il est utilisé pour fournir une meilleure interaction et expérience utilisateur en prenant en charge une navigation facile à travers les onglets et les listes déroulantes. Pour ajouter des actions à l'"appBar", créez un fichier XML dans le répertoire res / menu où vous définirez chaque action. Il est possible de définir les actions en Java, mais vous écrirez moins de code si vous utilisez XML

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

    <item

        android:id="@+id/ajout"
        android:title="Nouveau Contact"

        android:icon="@android:drawable/ic_menu_add"
        app:showAsAction="always"

    ></item>
</menu>
```

Ensuite, nous devons implémenter la méthode `onCreateOptionsMenu ()` dans notre activité.

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {

    getMenuInflater().inflate(R.menu.main_menu, menu);
    return super.onCreateOptionsMenu(menu);
}
```


Lorsque l'utilisateur clique sur une action , la méthode `onOptionsItemSelected ()` de l'activité est appelée. L'action peut être identifiée en appelant `"getItemId ()"`

SQLITE

Presque chaque application a besoin d'une place pour sauvegarder les données sur le long terme, plus longtemps que `savedInstanceState`. Android fournit un endroit pour le faire pour vous : un local système de fichiers sur votre téléphone ou tablette stockage en mémoire flash.

Presque chaque application a besoin d'une place pour sauvegarder les données sur le long terme, plus longtemps que `savedInstanceState`. Android fournit un endroit pour le faire pour vous : un local système de fichiers sur votre téléphone ou tablette stockage en mémoire flash. Android utilise des bases de données SQLite pour conserver les données Pourquoi SQLite ?

- Leger : La plupart des systèmes de base de données nécessitent un processus de serveur de base de données spécial pour que cela fonctionne. Pas SQLite, une base de données SQLite est juste un fichier. Lorsque vous n'utilisez pas la base de données, elle n'utilise pas temps processeur. C'est important sur un appareil mobile, car nous ne voulons pas vider la batterie
- Il est optimisé pour un utilisateur unique. Notre application est la seule chose qui va communiquer avec la base de données.



C'est stable et rapide. Les bases de données SQLite sont incroyablement stables. Ils peuvent gérer base de données, ce qui signifie que si vous mettez à jour plusieurs pièces de données et bousiller, SQLite peut restaurer les données. En outre, le code qui lit et écrit les données est écrit en code C optimisé.

Android crée automatiquement un dossier pour chaque application dans laquelle base de données peut être stocké. Lorsque nous créons une base de données pour l'application QUIZZ, il sera stocké dans le dossier suivant :
`/data/data/com.Moustapha.QUIZZ/databases`

Une application peut stocker plusieurs bases de données dans ce dossier. Chaque base de données se compose de deux fichiers. Le premier fichier est le fichier de base de données et porte le même nom en tant que base de données, par exemple «QUIZZ». C'est le principal Fichier de base de données SQLite. Toutes vos données sont stockées dans ce fichier. Le deuxième fichier est le fichier journal. Il a le même nom comme votre base de données, avec un suffixe de "-journal" - par exemple, «Journal QUIZZ». Le fichier journal contient toutes les modifications fait à votre base de données. En cas de problème, Android utilisera le journal pour annuler (ou annuler) vos dernières modifications.

Android vient SQLite Android utilise un ensemble de classes qui nous permet de gérer une Base de données SQLite. Il y a trois types d'objets qui font la gros de ce travail.

- SQLite Helper . Cette classe vous permet créer et gérer bases de données.
- Cursor, Un curseur vous permet lire et écrire à la base de données
- The SQLite Database qui nous permet d'avoir access à notre base de données

Création de l'assistant SQLite : Vous créez un assistant SQLite en créant une classe qui étend la classe SQLiteOpenHelper. Lorsque vous faites cela, vous devez surdéfinir les méthodes onCreate () et onUpgrade (). Ces méthodes sont obligatoires. La méthode onCreate () est appelée lorsque la base de données reçoit pour la première fois créée sur l'appareil. La méthode devrait inclure tout le code nécessaire pour créer les tables dont vous avez besoin pour votre application. La méthode onUpgrade () est appelée lorsque la base de données doit être mise à niveau. Par exemple, si vous devez apporter des modifications de table à .

L'assistant SQLite a besoin de deux informations : afin de créer la base de données. Premièrement, nous devons donner un nom à la base de données. En donnant la base de données un nom, nous nous assurons que la base de données reste sur le périphérique quand il est fermé. Si nous ne le faisons pas, la base de données ne sera créée que mémoire, donc une fois la base de données fermée, elle disparaîtra. La deuxième information que nous devons fournir est la version de la base de données. La version de la base de données doit être un entier valeur, à partir de 1. L'assistant SQLite utilise ce numéro de version pour déterminer si la base de données doit être mise à niveau. .

```
public BaseHelper(Context context){
    super(context,Tachedb.DB_NAME,
          null,Tachedb.DB_VERSION);
}
@Override
public void onCreate(SQLiteDatabase sqLiteDatabase) {

    String createTable = " CREATE TABLE " + Tache.table +"( " +
        Tache._ID + " INTEGER PRIMARY KEY  AUTOINCREMENT," + Tache.Col_Tache_Title + " TEXT NOT
    NULL);";

    sqLiteDatabase.execSQL(createTable);

}
```

L'assistant SQLite est chargé de créer la première base de données SQLite. le temps qu'il faut utiliser. Tout d'abord, une base de données vide est créée sur le périphérique, La méthode onCreate () d'assistance SQLite est ensuite appelée. La méthode onCreate () reçoit un objet SQLiteDatabase en tant que paramètre. Nous pouvons utiliser cela pour exécuter notre commande SQL avec la méthode : SQLiteDatabase.execSQL (String sql) ;

La classe SQLiteDatabase contient plusieurs méthodes qui vous permettent d'insérer, mettre à jour et supprimer des données. Nous examinerons ces méthodes au cours des prochaines pages. en commençant par l'insertion de données.

Si vous devez pré-remplir une table SQLite avec des données, vous pouvez utiliser le Méthode SQLiteDatabase insert (). Cette méthode vous permet d'insérer données dans la base de données et renvoie l'ID de l'enregistrement une fois qu'il a été inséré. Si la méthode ne parvient pas à insérer l'enregistrement, elle renvoie la valeur -1.

```
SQLiteDatabase db = sqlHelper.getWritableDatabase();

    ContentValues values = new ContentValues();

    values.put(Score.columnName, editText.getText().toString());

    values.put(Score.columnScore, score);

    long insertion = db.insert(Score.tableName, null, values);

    if (insertion == -1)
    {
        Toast.makeText(MainActivity.this, "Insertion
failed", Toast.LENGTH_SHORT).show();
    }

    else
    {
        Toast.makeText(MainActivity.this, "Insertion with
success", Toast.LENGTH_SHORT).show();
    }
}
```

Obtenir des données de la base de données avec un curseur Un curseur vous donne accès enregistrements de la base de données. Vous spécifiez quelles données vous voulez accéder, et le curseur ramène le enregistrements de la base de données. Vous naviguez ensuite dans les enregistrements fourni par le curseur



```
Cursor cursor = db.query(Tache.table,  
    new String[]{  
        Tache._ID,Tache.Col_Tache_Title  
    },null , null,  
    null,null,null);
```


Avec une condition

```
Cursor cursor = db.query(Score.tableName,new String[]{
    Score.columnName,Score.columnScore
}, "nom= ? or nom =?",new String[]{
    "dji by",
    "moustapha"

},null,null, Score.columnName
);
```

Listview

L'affichage des éléments d'une liste est un modèle très courant dans les applications mobiles. L'utilisateur voit une liste d'éléments et peut les faire défiler. Android fournit les classes `ListView` et `ExpandableListView` capables d'afficher une liste d'éléments pouvant défiler. La classe `ExpandableListView` prend en charge un groupe d'éléments.

Adapter

Afin de remplir une ListView, il existe un objet nommé Adapter, qui va prendre en entrée une liste d'objets et un layout XML et va générer pour chaque entrée une cellule formatée.

Adapter

Lisview

Pour créer une liste plus complexe Il faut maintenant créer un layout pour nos cellules. L'étape suivante est assez simple à deviner : essayer de reproduire le design que vous voulez pour chaque ligne de votre liste !

Nous allons ne garder que le principal pour chapitre, chez nous une ligne sera composée de :
un avatar (que nous réduirons à un simple carré de couleur) le nom du joueur et son score

Lisview

Afin de réduire la consommation en mémoire de notre ListView, le principe est le suivant : la ListView ne stock pas plus de vues qu'elle a la capacité d'en afficher, dès qu'une vue sors de l'écran (au scroll) elle va être réutilisée afin de produire la nouvelle vue à apparaître. On dit alors que nos vues sont recyclées. Afin d'éviter d'appeler les méthodes findViewById à chaque réutilisation des vues, Android a rajouté un concept, nommé ViewHolder (gardien/protecteur de vue), qui va avoir le rôle de mini contrôleur, associé à chaque cellule, et qui va stocker les références vers nos sous vues (dans notre cas : titre, texte et image). Et va s'associer à une vue.

```
public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {  
  
    Message message = getItem(position);  
  
    if(convertView==null)  
    {  
        convertView = LayoutInflater.from(getContext()).inflate(R.layout.messagerow,parent,false);  
    }  
  
    TextView auteur = convertView.findViewById(R.id.auteur);  
  
    TextView messageText = convertView.findViewById(R.id.messagef);  
  
    auteur.setText(message.getAuteur());  
  
    messageText.setText(message.getMessage());  
  
    return convertView;  
  
}
```

```
void whenAppfinish()
{
    final EditText editText = new EditText(this);
    AlertDialog dialog = new AlertDialog.Builder(this)
        .setTitle("Game over")
        .setMessage("Veuillez inserer votre prenom")
        .setView(editText)
        .setPositiveButton("Inserer", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {

            }
        })
        .setNegativeButton("annuler",null)
        .create();

    dialog.show();
}
```